Length-Targeted Generation of Equivalence-Checked Program Clones for Software Diversity

Benita Kingsley Prof. Agadakos
Montclair State University
kingsleyprib1@montclair.edu

Abstract

Software monoculture means one exploit generalises across every deployment of a program. N-version diversity counters this by shipping structurally distinct binaries that behave identically, but producing those variants by hand does not scale. We present the clone-generation layer of AXIOM, a system that produces program variants which are structurally different, behaviourally identical, and close to a caller-specified size. The layer composes a catalogue of 17 semantics-preserving transformations under a length-targeted beam search adapted from Auto-SPT, and gates every candidate through a four-layer equivalence oracle built on recorded golden vectors rather than tests alone. Across 24 (module, target-size) cells on a four-module industrial control corpus, the search explored 2,398 candidates and returned an oracle-verified variant in 24/24 cells, with control-flow structure altered in 16/24. On a second, independently authored corpus the layer diversified 12 of 18 runs with 11 accepted by the oracle, restructuring control flow entirely on three of six programs, though a seeding defect in that build means those runs do not yet demonstrate migration and diversification composed end to end. Growing the catalogue from 12 to 17 rules raised mean rename-insensitive diversity by 73% and improved length accuracy. An ablation on the ranking metric changed the selected variant in only 3/24 cells and never changed length accuracy, which we trace to the lexicographic objective. We report these as preliminary results and are explicit about what the oracle does and does not establish.

1 Introduction

A vulnerability in a widely deployed program is a vulnerability in every installation of it. This is the software monoculture problem, and the classical answer is N-version programming: run several implementations that agree on behaviour but differ in structure, so that a memory-corruption exploit tuned to one layout fails on the others. The obstacle has always been cost. Writing independent implementations is expensive, and automated diversification has historically operated at the binary or compiler level, where the transformations are opaque to the developer and hard to audit.

This paper describes the component of AXIOM that generates such variants at the *source* level. Three properties make the problem non-trivial:

1. **Equivalence must be checked, not assumed.** A transformation that is semantics-preserving in principle can still be misapplied. Program equivalence is undecidable in general (Rice's theorem), so we do not claim proof; we claim equivalence *with respect to a stated set of checks*, and we state them.
2. **Diversity must survive the obvious objection.** A variant produced by renaming identifiers is not diverse in any useful sense. Our ranking metric is computed after discarding every identifier, so renaming cannot inflate it.
3. **Size is a deployment constraint.** A variant that is three times the size of the original may not fit the target. We treat a caller-supplied line target as the primary search objective, with diversity as a tie-break — not the reverse.

This is a preliminary report on one layer of a larger system. We report what we measured, and Section 4 states plainly what these numbers do not establish.

2 Approach

2.1 Transformation catalogue

The catalogue holds 17 semantics-preserving transformations (SPTs), of which the four LLM-generated rules are opt-in: they are loaded only when the caller asks for them, so the standalone sweeps of Section 3.3 use all 17 while the pipeline deployment of Section 3.5 used the 13 hand-written rules alone. Thirteen are hand-written using LibCST, a format-preserving concrete syntax tree library: transforming through a CST rather than an AST means every part of the file the transformation did not touch comes back byte-identical, so a reviewer reads a minimal diff instead of a whole-file reformat. Four further rules were LLM-generated from a specification and then subjected to the same oracle as every other rule.

Rules fall into three groups. *Statement-level* rules change expression structure without touching control flow (`split_return_expression`, `extract_subexpression`, `expand_aug_assign`). *Control-flow* rules change the branch graph (`ternary_to_if_else`, `invert_if_else`, `demorgan_branch_swap`, `for_range_to_while`,

```
def level(counts):
    return (0.0 if counts < 0
            else counts * 0.0025)
```

Listing 1: Original (top) and variant (bottom) after `ternary_to_if_else` then `split_return_expression`.

```
def level(counts):
    if counts < 0:
        return 0.0
    scaled = counts * 0.0025
    return scaled
```

Metric	Value	Reduces the code to
`ncd`	0.3600	compressed bytes
`ast_structural`	0.2500	ordered node types
`du_chain`	0.5000	define–use pairs
`cfg_shape`	1.0000	branches by depth
`composite`	0.6250	

Figure 1: The same edit, measured five ways.

`merge_nested_if`). *Binding* rules change names and temporaries (`rename_locals`, `inline_single_use_return`).

The distinction matters because the two groups move different metrics, and a catalogue weighted only toward the first produces variants that are longer but structurally unchanged — a failure mode we observed directly and address in Section 3.3.

Listing 1 makes the metrics concrete. `cfg_shape` reaches 1.0 because the original contains no branch construct at all ($\varnothing$) while the variant contains one ({`If@0: 1`}), so the Jaccard intersection is empty. `ast_structural` reports only 0.25 over the same edit, because 17 of the node types still align in order. The two are not in conflict — they answer different questions, which is why we report several rather than one, and why `composite` weights both equally.

2.2 Four-layer equivalence oracle

Every candidate is checked before it may enter the beam. All four layers must pass, evaluated cheapest-first in a scratch copy of the program, in a subprocess, so a candidate that hangs or corrupts state cannot affect the next:

1. **Import integrity** — every first-party module still imports.
2. **Golden regression** — every probe reproduces its recorded value *exactly*. Tolerance is zero; float reassociation is therefore forbidden, which is deliberate for control code.
3. **Invariants** — properties that must hold for *any* input, swept over ranges rather than checked at points.
4. **Coverage** — the program's own test suite still passes.

Golden vectors are recorded once by executing the *pristine* program and frozen. They are never re-recorded after a rewrite: an oracle that re-derives its answer key from the code under test agrees with whatever it is given, which is the one failure mode a golden-vector scheme must not have. Probes are chosen for branch coverage rather than representativeness, since a golden set that never enters a fault path cannot detect a broken fault path.

Invariants are validated against the pristine program *before* being trusted. This caught a real defect in our own harness: an early invariant asserted that a counts-to-level conversion clamped to $[0, 10]$, which the original does not do — it returns -1.22 for -500 counts. An invariant the original violates rejects a perfect copy, and the oracle would have looked rigorous while accepting nothing.

2.3 Diversity metrics

We report five metrics, of which two are rename-insensitive and are the only ones we make claims from.

`ast_structural` is the ranking signal: the pre-order sequence of AST node *types*, with all identifiers discarded, compared by sequence alignment, reported as $1 - \text{ratio}$. A histogram of node types proved far too insensitive at this scale — extracting a subexpression from an 89-line module moved a cosine distance by $\sim$0.0003, which cannot rank anything — so the comparison is order-aware.

`cfg_shape` counts control-flow constructs tagged by nesting depth (`If@0`, `For@1`) and takes a weighted Jaccard distance. It is identifier-free, so branch restructuring registers and renaming does not.

`composite` is $0.5 \cdot$ `ast_structural` $+ 0.5 \cdot$ `cfg_shape`. The equal weighting is deliberate: `ast_structural` alone rewards statement-level padding without ever altering control flow.

`ncd` is normalised compression distance [3], used unmodified with zlib as the compressor. `du_chain` is a Jaccard distance [4] over define–use pairs. Both are explicitly rename-sensitive and we do not rank by them.

Provenance of these metrics. The ingredients are established; the specific measures are ours, and we state which is which because the distinction affects how much weight the numbers can carry. Normalised compression distance and the Jaccard index are standard and used as published. `ast_structural` composes two standard pieces — AST node-type sequences and Ratcliff/Obershelp sequence matching [5] — but the composition is our own. `du_chain` approximates define–use chains by collecting *(function, used-name)* pairs rather than performing reaching-definitions analysis. `cfg_shape` is not a control-flow graph in the formal sense: it has no basic blocks and no edges, only a depth-tagged histogram of branch and loop constructs. `composite` weights its two terms equally by judgement, not by fitting.

Consequently none of these is validated against an external referent. We can show that a variant differs structurally by these measures; we cannot yet show that the

difference predicts anything an attacker or a clone detector would care about. Correlating them against an independent clone detector — the objective Auto-SPT optimises directly — is the cheapest way to ground them and is planned work.

2.4 Length-targeted beam search

The search skeleton is Auto-SPT's Algorithm 1 — maintain a beam of the B best candidates and iterate expand, filter, select — with its clone-detector objective replaced by a size objective. Each iteration applies every rule to every beam member, discards candidates that are not valid Python, submits the remainder to the oracle, and sorts the survivors by

$$\text{key}(c) = \big(|\text{lines}(c) - T|,\ -D(x, c) \big) \tag{1}$$

where T is the requested line target and D the ranking metric. The key is a tuple compared lexicographically. Diversity can therefore never buy back a length miss; it only orders candidates already tied on length. This is what keeps the length control honest, and it is the substantive departure from Auto-SPT, whose objective is a clone-detector score.

The search stops when

$$|\text{lines}(c^*) - T| \ \le\ \max\big(1,\ \lceil |T - \text{lines}(x)| \cdot \tau \rceil\big) \tag{2}$$

with $\tau = 0.10$. Tolerance scales with the requested *change*, not the file size. Sizing it off the absolute count makes it meaningless: a 10% band on a 97-line target is ±9 lines, so a search asked to add 8 lines could add none and stop satisfied.

3 Evaluation

3.1 Corpora

The primary corpus is four industrial-control modules (89–134 LOC): a PID governor, a pump controller, a protective relay, and a sensor pipeline. Its oracle holds 1,904 recorded observations across the four modules.

The second corpus is an independently authored set of twelve industrial-control programs in four categories, of which categories C and D specify that first-party logic must be diversified in addition to any library substitution. Categories A and B specify substitution only, with no logic mutation, and are therefore outside this layer's scope; the six C/D programs are the ones this paper uses. We authored a golden-vector oracle for those six, holding 170 recorded observations, following the same four-layer construction described in Section 2.2. This corpus is the deployment check of Section 3.5: it was not written by us, and it exercises the layer inside the full pipeline rather than standalone.

Each module is swept at six size ratios (0.85, 0.90, 1.00, 1.10, 1.20, 1.30 of original length), giving 24 (module, target) cells. Beam size 3, at most 6 iterations.

Table 1: Per-module results over 24 cells. Diversity values are means over the six size targets; `cfg`>0 counts cells whose winning variant altered control-flow structure at all.

Module	LOC	Cand.	`ast`	`cfg`	`cfg`>0
pid_governor	89	539	0.0616	0.1429	3/6
pump_controller	96	735	0.0522	0.1524	5/6
protective_relay	134	594	0.1000	0.0556	3/6
sensor_pipeline	114	530	0.0573	0.1528	5/6
All		2398	0.0678	0.1259	16/24

Table 2: Effect of catalogue size. Identical corpus, identical 24 cells, identical search parameters.

Catalogue	`ast`	Cand.	Mean |err|	On target
12 rules	0.0392	1402	12.50	4/24
16 rules	0.0652	2160	10.08	4/24
17 rules	0.0678	2398	8.88	5/24

3.2 Main results

Table 1 reports the sweep. The search explored 2,398 candidates and returned an oracle-verified variant in **24/24** cells, in 395 s of total search time (4.83 iterations per cell on average).

Two figures deserve comment rather than celebration. First, all 2,398 candidates that reached the oracle passed it. This is what one expects when the transformations are semantics-preserving by construction, but it does mean the sweep provides no evidence that the oracle *discriminates*. We establish that separately: injecting a deliberate fault into an otherwise-correct module is rejected at the import layer, and in pipeline deployment the same oracle rejected 3 of 18 runs (Section 3.5). Second, only 5/24 cells hit their line target exactly, with a mean absolute error of 8.88 lines. Length control is best-effort, and we report the miss rather than reporting only the cells that landed.

`protective_relay` is the clearest negative result: highest `ast_structural` (0.1000) but lowest `cfg_shape` (0.0556) and the worst length error. It has no shrink path in the current catalogue — only two rules can remove lines — so every sub-unity ratio for that module is unreachable.

3.3 Catalogue growth

Table 2 isolates the effect of the catalogue. Adding the four LLM-generated rules and then one hand-written rule raised mean rename-insensitive diversity by **73%** and cut mean length error from 12.50 to 8.88 lines. The gain is superlinear in rule count because rules compose: each new rule multiplies the reachable space rather than adding to it.

The single hand-written addition, `for_range_to_while`, was motivated by a specific failure. Under the 12-rule

Table 3: Second corpus, three seeds per program, running inside the full pipeline. `Div.` counts seeds in which the agent applied at least one transformation; `Eq.` counts those the oracle then accepted. The corpus's other six programs (categories A and B) carry no logic-mutation requirement, so the pipeline does not route them to this layer and they contribute none of the 18 runs; they are absent here because the layer was never invoked on them, not because the results were poor.

Program	Cat	Div.	Eq.	`comp`	`cfg`
C1_pid_pump	C	0/3	—	—	—
C2_tank_fsm	C	0/3	—	—	—
C3_anomaly	C	3/3	3/3	0.5448	1.0000
D1_boiler	D	3/3	2/3	0.2110	0.3333
D2_pump_station	D	3/3	3/3	0.5529	1.0000
D3_telemetry	D	3/3	3/3	0.5438	1.0000
All		12/18	11/12	0.4631	0.8333

catalogue most winning variants scored `cfg_shape` = 0 — structurally identical control flow — which undermines the entire premise. An earlier attempt, `elif_to_nested_if`, scored exactly 0 because `elif` and `else: if` are indistinguishable in the Python AST. Rewriting counted `for` loops as explicit `while` loops does change the graph, and control-flow structure is now altered in 16/24 cells.

3.4 Ranking-metric ablation

We re-ran the full sweep with the ranking metric changed from `ast_structural` to `composite`, holding everything else fixed. The selected variant differed in **3 of 24** cells, and achieved line counts were *identical in 24/24*.

This is the behaviour Equation 1 predicts: because the key is lexicographic and length dominates, changing the ranking metric can only reorder candidates that are already tied on length. The bottleneck on this corpus is the reachable set — what the catalogue can produce — not the choice among what it produces. We report this as a negative result. It also means the metric choice is safe to revisit later without invalidating length-accuracy claims.

3.5 Deployment inside the full pipeline

The layer runs as an agent inside AXIOM's migration pipeline, invoked on first-party logic after a library substitution. These runs used the 13 hand-written rules only; the four generated rules are opt-in and the pipeline did not request them, so this deployment exercises a smaller catalogue than Table 2's best row. Table 3 reports the six C/D programs of the second corpus at three seeds each. A defect in how the search was seeded — described at the end of this section — means these runs diversified the pristine modules rather than the migrated ones; we report them for what they do measure and are explicit about what they do not.

Table 4: The twelve-program corpus split into ten ordinary migrations and two negative controls, three seeds each. A migration counts as correct when the substitution completed, the source actually changed, and the program's tests passed; a control counts as correct when the substitution was *not* completed. "w/ reason" counts controls declined with a recorded stage failure rather than silently.

Configuration	Migr. (30)	Ctrl (6)	w/ reason (6)	Overall (36)
single_shot	20/30	0/6	0/6	20/36
proteus_no_verif	12/30	6/6	0/6	18/36
proteus_full	18/30	6/6	**6/6**	**24/36**

Three results matter here. First, **three of six programs reached `cfg_shape` = 1.0** — control-flow structure entirely restructured, with every recorded probe value still reproducing exactly. Mean composite diversity on this corpus (0.4631) is substantially higher than on the primary corpus (0.0968), because these programs are smaller (25–38 LOC), so each applied rule moves a larger fraction of the structure. That is a property of program size, not evidence that the layer works better here.

Second, the **single oracle rejection** — one D1 seed of three — is the discriminating evidence the main sweep lacks. The same rule chain that the oracle accepted on two seeds it rejected on the third.

Third, C1 and C2 produced **no transformations at all**. Both are cases where the specified target is not an installable package but a reimplementation, and the upstream resolver stage returns no substitution to diversify; the layer is never reached with anything to work on. This is a pipeline-integration limit rather than a limit of the catalogue.

Refusal as a measurable outcome. Two further programs in the same corpus specify a migration that cannot be written: both ask for `pymodbus`→`minimalmodbus`, but the originals speak Modbus over TCP and `minimalmodbus` is serial-only, exposing no TCP entry point at all. The correct behaviour is to decline.

Scored as ordinary tasks these two punish the only right answer, so we score them as negative controls: correct means *not* completing the substitution. Table 4 separates the two populations.

The full pipeline declined all six control runs with a recorded stage failure — three at the substitution agent, three at the verifier. The no-verification ablation also declined all six, but recorded no reason for any of them, which is a materially weaker result: an arm that does nothing on every program would score full marks on negative controls, so outcome alone is cheap credit and the "w/ reason" column is what separates deliberate refusal from inactivity. The single-prompt baseline completed

the impossible substitution in all six runs, and our un-injected factory check flags five of those six as raising on first real call.

We report this because a system that refuses cannot demonstrate that refusal in a success-rate table — a correct decline and an outright failure are indistinguishable there. It is also the clearest evidence in this paper that the checking apparatus rejects rather than rubber-stamps. Two caveats: this is a re-scoring of existing runs rather than new data, and the criterion was chosen after observing the results. It rests on a pre-existing column ("did the substitution complete"), and the impossibility is verifiable from the target library's own signature, neither of which depends on how any arm scored.

A seeding defect these runs exposed. The numbers in Table 3 were produced by a build in which the search seeded from the corpus's *pristine* module rather than from the patched module handed to it by the preceding stage. Two consequences follow. The library substitution was discarded, so the variants measured here are diversified originals rather than diversified migrations. And the line target, computed from the patched text, described a different file than the one searched — in the worst case a 25-line module was given a 167-line target and reported a 138-line miss, which is why we report no length-accuracy figures for this corpus. We have since fixed the seeding; on a re-run of one program the search now starts from the patched source, retains the substitution, and lands within one line of target. The diversity and equivalence results in Table 3 stand as measurements of the catalogue and oracle, but they do not yet demonstrate length control or migration-plus-diversification composed end to end. That composition is the first experiment we intend to repeat.

4 Limitations

We do not prove equivalence. Program equivalence is undecidable. Every claim here is equivalence with respect to four stated checks. A rewrite that breaks a path neither the probes nor the tests visit would pass.

The oracle inherits its corpus's blind spots. Both corpora inject test doubles at construction sites, so probes that follow the same pattern do not execute the un-injected path. We are aware of this and treat it as future work rather than a solved problem.

The metrics are not externally validated. Two of the five are standard and used as published; `ast_structural`, `cfg_shape` and `composite` are our own constructions, and nothing yet ties them to an outcome an attacker or a clone detector would recognise. A high `cfg_shape` means the branch histogram moved, no more than that.

Scale. Four modules and 24 cells is a pilot; the second corpus adds six programs. The 73% and 16/24 figures are measurements on the primary corpus, not general claims, and the two corpora differ enough in program size that their diversity values are not directly comparable.

Shrinking is weak. Only two rules remove lines, so sub-unity targets are frequently unreachable, which dominates the 8.88-line mean error.

No attack evaluation. We measure structural difference, not exploit resistance. Whether these variants defeat a real memory-corruption exploit is the obvious next experiment and is not attempted here.

5 Related work

Auto-SPT [1] provides the beam-search skeleton and the notion of composing semantics-preserving transformations under a search objective; we replace its clone-detector objective with a size target, which changes what the search optimises for. Classical N-version programming motivates the goal; compiler-level and binary-level diversification pursue it at a layer where the transformation is not auditable by the developer. MigrateLib [2] informs our use of format-preserving rewriting for minimal diffs.

6 Conclusion

We described AXIOM's clone-generation layer and reported preliminary results: an oracle-verified variant in 24/24 cells over 2,398 explored candidates, control flow altered in 16/24, a 73% diversity gain from growing the catalogue from 12 to 17 rules, and a ranking-metric ablation that returned a clean negative result. The immediate next steps are shrink-capable transformations, probes that exercise un-injected construction paths, and an attack-based evaluation to test whether measured structural difference translates into exploit resistance.

References

[1] Hooda et al. Auto-SPT: automated semantics-preserving transformations. *arXiv:2512.06042*, 2025.

[2] Islam, Jha, Mahmoud, and Nadi. MigrateLib: library migration with format-preserving rewriting. *arXiv:2510.08810*, 2026.

[3] R. Cilibrasi and P. M. B. Vitányi. Clustering by compression. *IEEE Transactions on Information Theory*, 51(4):1523–1545, 2005.

[4] P. Jaccard. Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37:547–579, 1901.

[5] J. W. Ratcliff and D. E. Metzener. Pattern matching: the gestalt approach. *Dr. Dobb's Journal*, 13(7):46–72, 1988.